

12

Neural Networks

Data Mining II | MGI 2526

May 19 and 20, 2026

Carina Albuquerque

calbuquerque@novaims.unl.pt

Agenda

01



**Introduction to
Neural
Networks**

02



**Hyperparameters
in NN**

03



**The perceptron
- Example**

04

**Multilayer
Perceptron**



01.

Introduction to Neural Networks

1. Neural Networks

Introduction

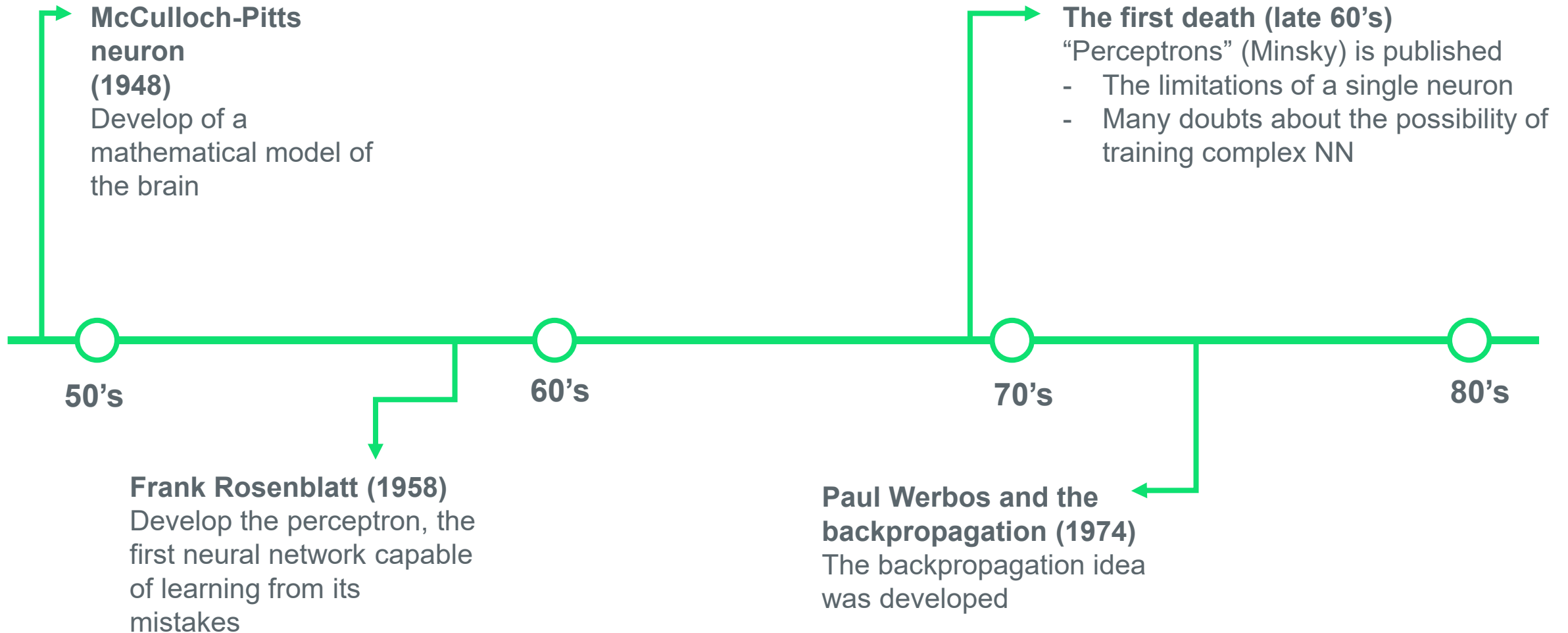


What are Neural Networks?

A type of machine learning model that mimic the **structure and function of the human brain**, using layers of interconnected nodes to process and analyze data.

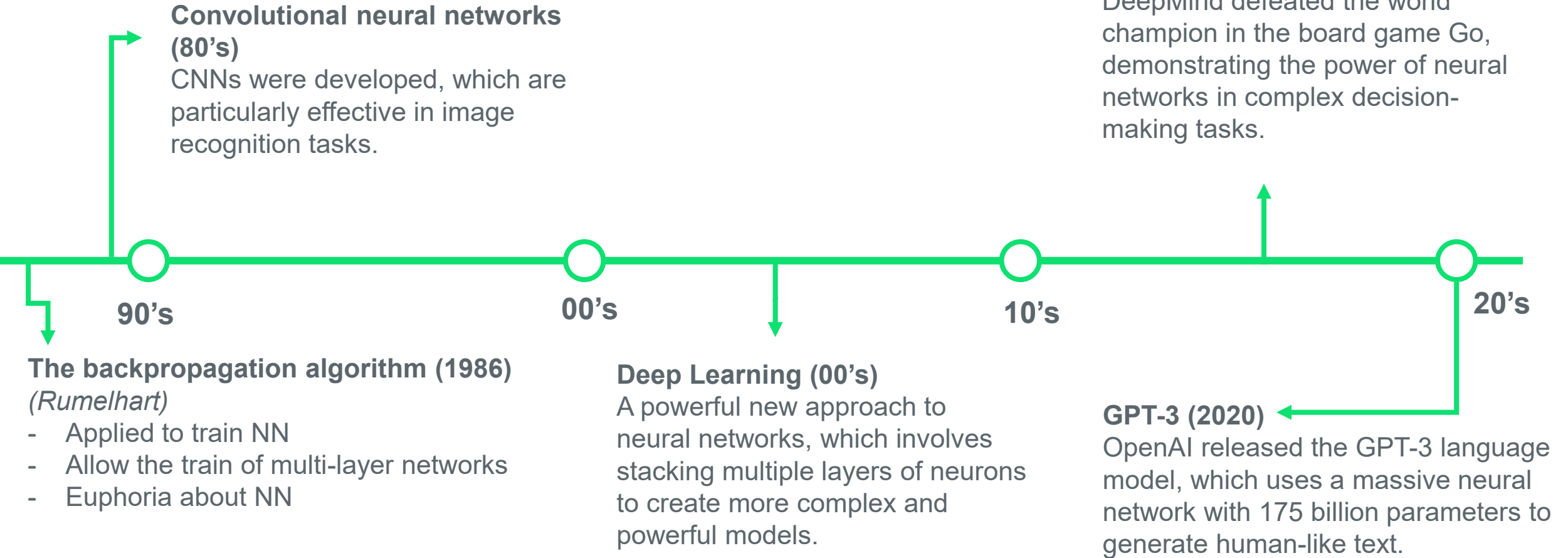
1.1. Neural Networks

The idea behind neural networks - history



1.1. Neural Networks

The idea behind neural networks - history



1.1. Neural Networks

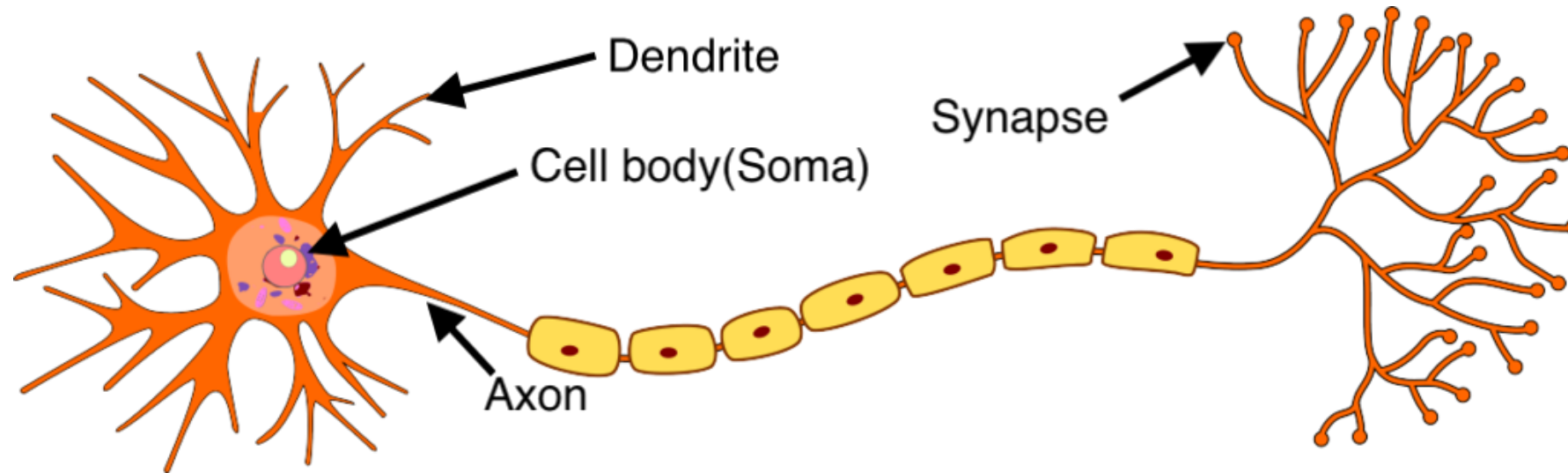
The idea behind neural networks - history



https://youtu.be/FwFduRA_L6Q

1.1. Neural Networks

The idea behind neural networks – The biological neuron



Dentrite – Receive information

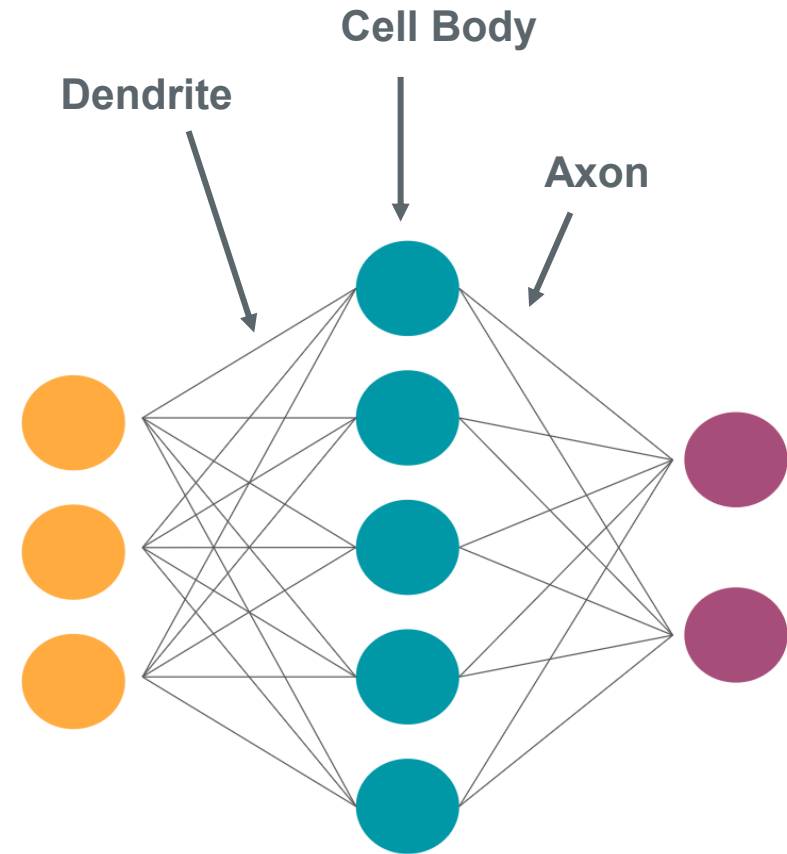
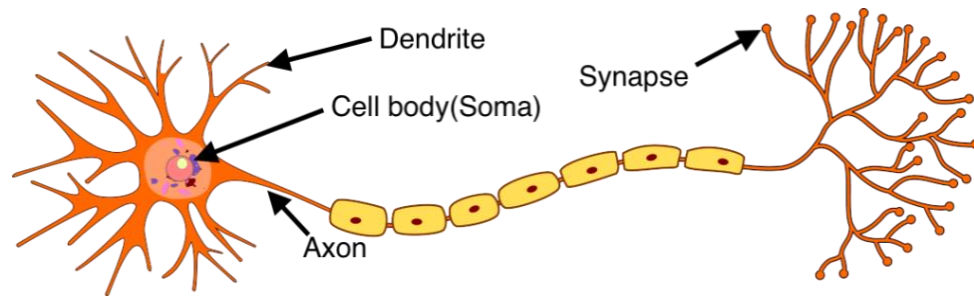
Cell Body – Process Information

Axon – Carries processed information to other neurons

Synapse – Junction between axon and dendrites of other neurons

1.1. Neural Networks

The idea behind neural networks – The biological neuron



1.2. Neural Networks

The perceptron



**What is a
Perceptron?**

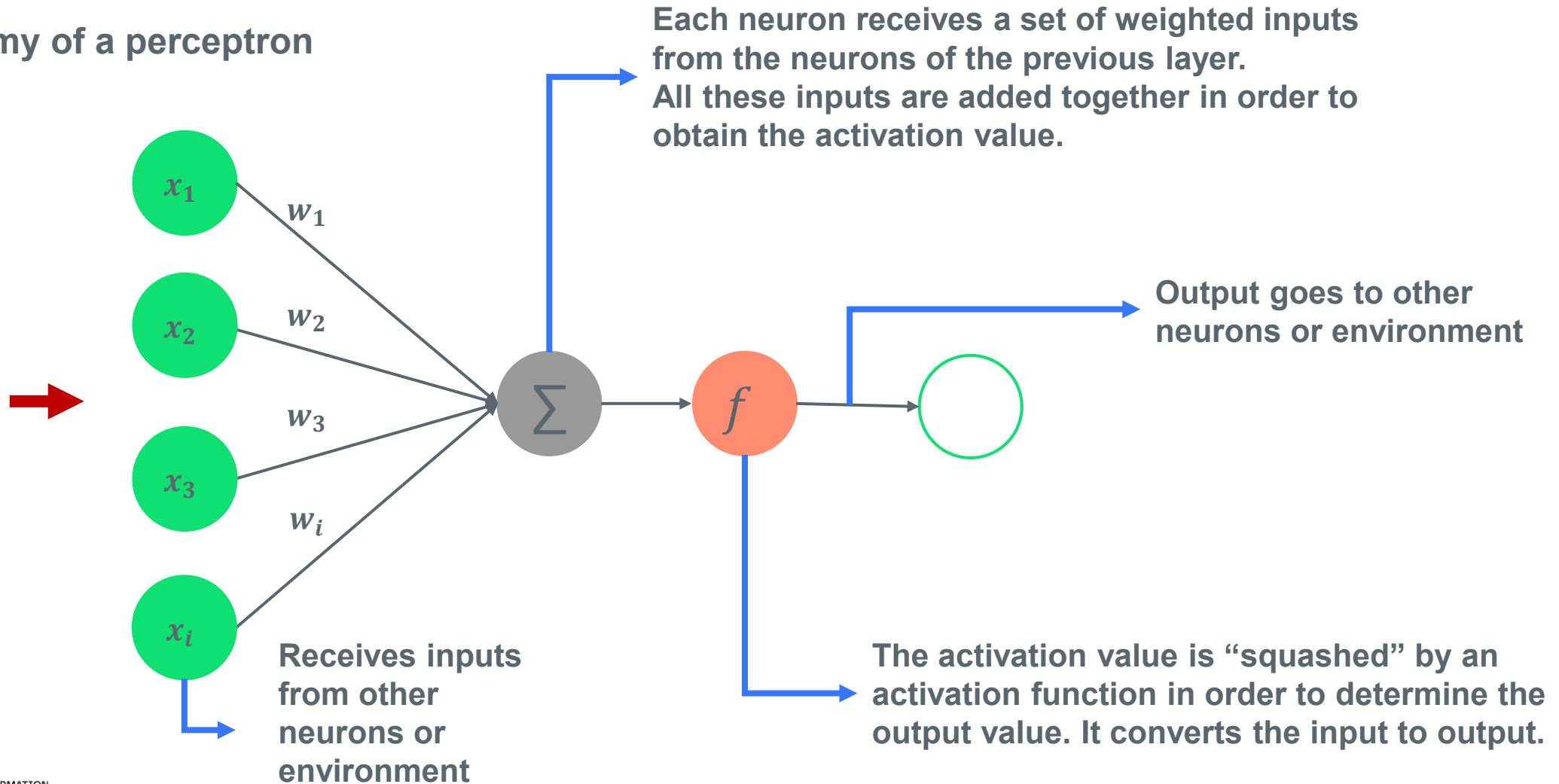
A type of neural network
composed by a single layer of interconnected nodes
that can learn to recognize patterns in data
by adjusting the weights of the connections between neurons.

1.2. Neural Networks

The perceptron

The anatomy of a perceptron

X1	X2	X3	X4	T
0.39	0.66	0.8	0.22	0
0.55	0.28	0.81	0.21	1
0.38	0.94	0.52	0.83	0
0.2	0.98	0.16	0.72	0
0.03	0.46	0.03	0.81	0
0.41	0.01	0.87	0.97	1
0.97	0.08	0.8	0.09	0
0.9	0.9	0.84	0.07	1
0.01	0.38	0.56	0.41	0
0.64	0.94	0.67	0.94	0
0.17	0.13	0.64	0.12	1
0.74	0.46	0.48	0.5	1
0.82	0.99	0.43	0.68	0
0.2	0.99	0.1	0.88	1



1.2. Neural Networks

The perceptron – How does it work?

- A perceptron receives multiple input signals, and if the sum of the input signals exceed a certain threshold, it either returns a signal or remains silent otherwise;

How does a perceptron learn?

1. A perceptron makes a prediction;
2. It is evaluated how well the prediction was.
 1. Measure the error
 2. Backpropagate the error to adjust the weights

1.2. Neural Networks

The perceptron – How does it work?

- The weights are initialized at random at the beginning of the training
- For each element of the training set, the error is calculated with the difference between the desired and the actual output. The error calculated is used to adjust the weights
- The process is repeated until:
 - The error level over the training set reaches a specified threshold, or
 - A maximum number of iterations is achieved
- This iteration allows the perceptron to change the weights in order to predict more accurately.

Learning in NN is a search problem

Search for the best possible configuration of weights so the network's error falls as closer to 0 as possible

1.2. Neural Networks

The perceptron – How does it work?

How to change the weights?

1. Define the error:

$$error = y_{target} - y_{prediction}$$

2. Define the hyperparameter learning rate (α):

A scaling factor that determines how large or smooth the weight vector updates should be

3. Calculate the new weights:

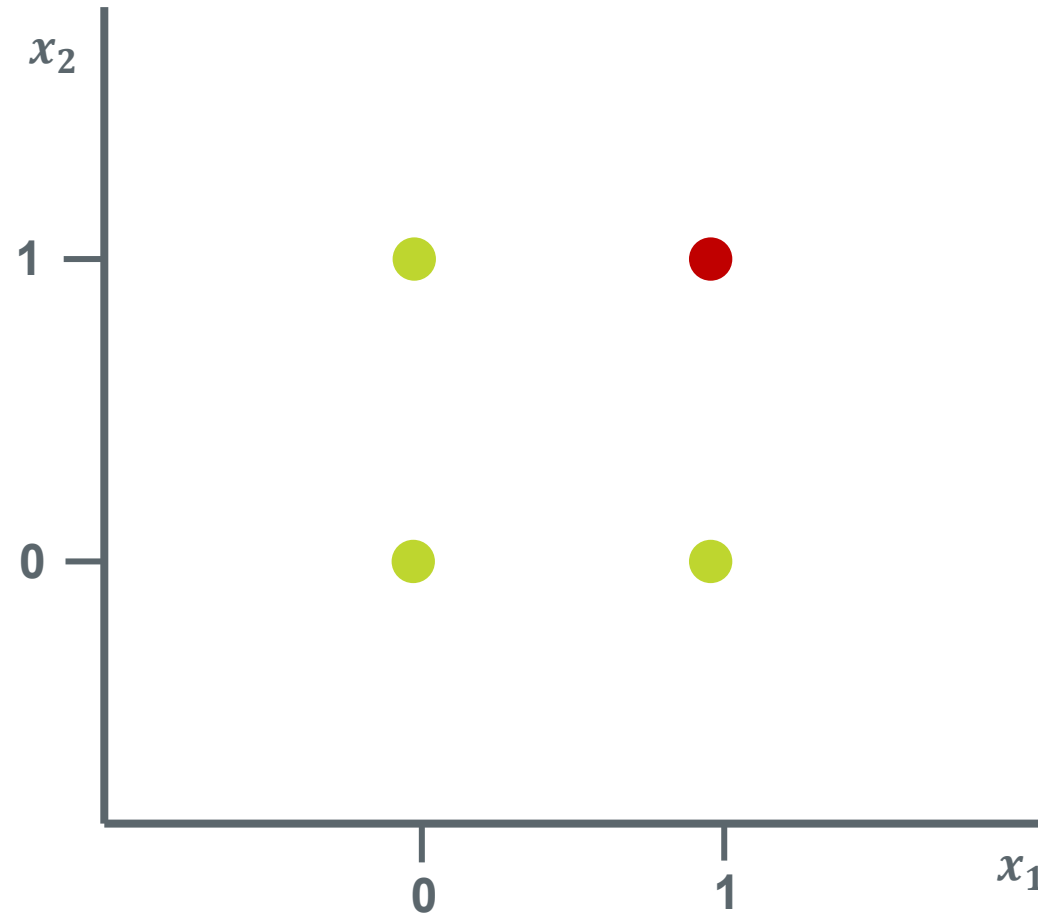
$$Weight_{new} = Weight_{old} + (\alpha \times error \times x_1)$$

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem



1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Define random weights

$$w_1 = 0.9$$

$$w_2 = 0.9$$

2. Define a threshold and the learning rate

$$t = 0.5$$

$$\alpha = 0.5$$

3. Apply first instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.9 + 0 \times 0.9 = 0$$

4. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

5. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.9$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply second instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.9 + 1 \times 0.9 = 0.9$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 0 - 1 = -1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.9 + (0.5 \times -1 \times 0) = 0.9 - 0 = 0.9$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.9$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply third instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.9 + 0 \times 0.4 = 0.9$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 0 - 1 = -1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.4 + (0.5 \times -1 \times 0) = 0.4 - 0 = 0.4$$

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply fourth instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.4 + 1 \times 0.4 = 0.8$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply first instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.4 + 0 \times 0.4 = 0$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply second instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.4 + 1 \times 0.4 = 0.4$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

AND Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply third instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.4 + 0 \times 0.4 = 0.4$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

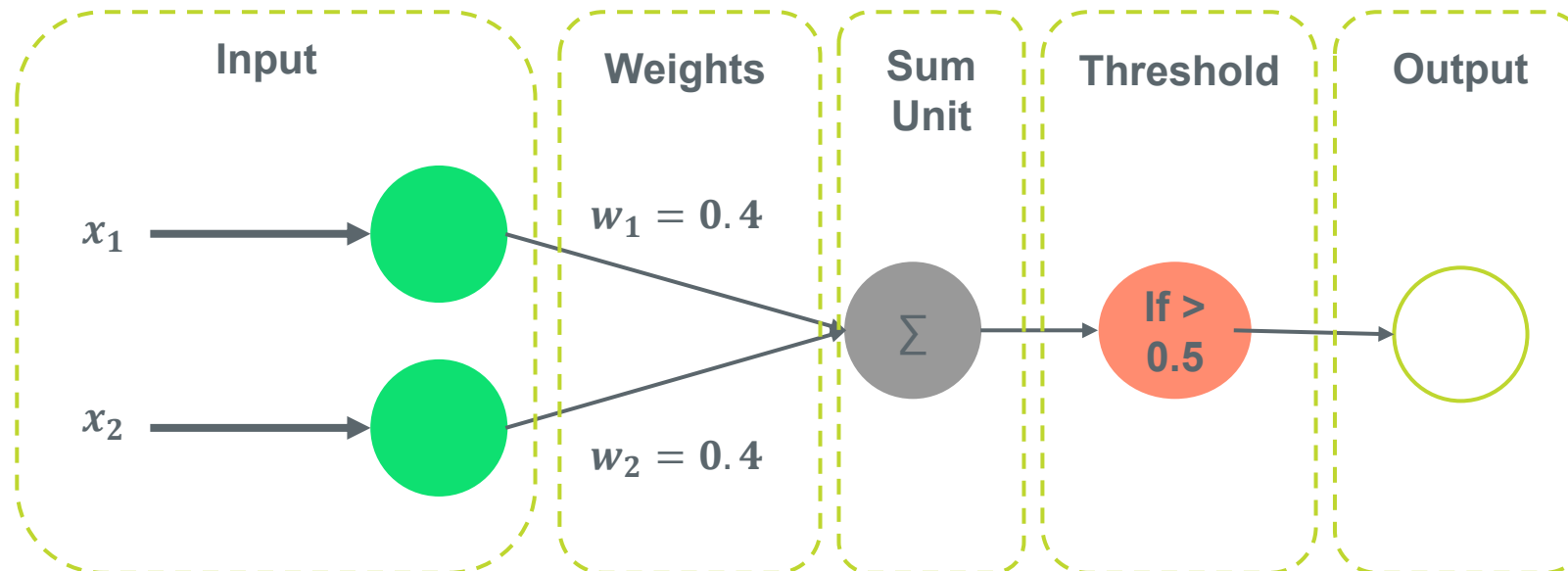
$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error
5. All instances are correctly classified. Finish here.

1.2. Neural Networks

The perceptron – An example step by step: AND Problem

The anatomy of a perceptron

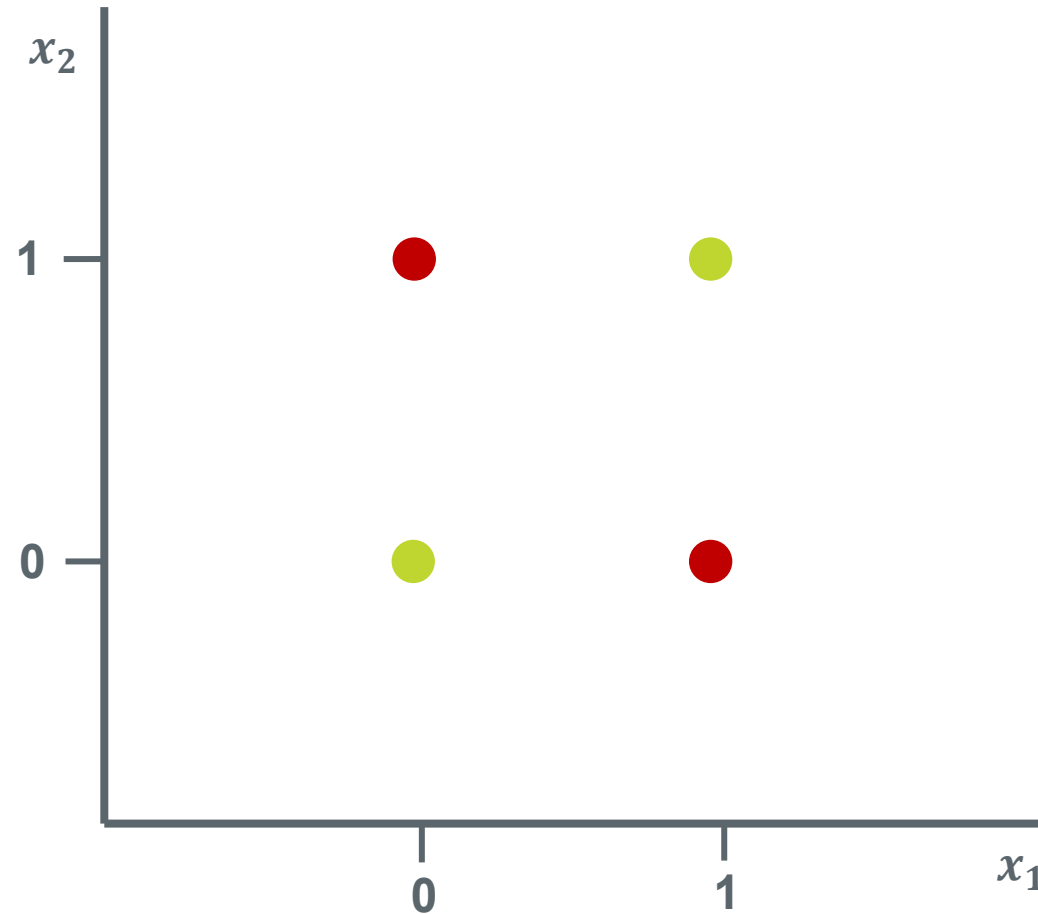


1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem



1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Define random weights

$$w_1 = 0.9$$

$$w_2 = 0.9$$

2. Define a threshold and the learning rate

$$t = 0.5$$

$$\alpha = 0.5$$

3. Apply first instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.9 + 0 \times 0.9 = 0$$

4. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

5. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weights

$$w_1 = 0.9$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply second instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.9 + 1 \times 0.9 = 0.9$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weight

$$w_1 = 0.9$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply third instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.9 + 0 \times 0.9 = 0.9$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weight

$$w_1 = 0.9$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply fourth instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.9 + 1 \times 0.9 = 1.8$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 0 - 1 = -1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply first instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.4 + 0 \times 0.4 = 0$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weights and the threshold

$$w_1 = 0.4$$

$$w_2 = 0.4$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply second instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 0 \times 0.4 + 1 \times 0.4 = 0.4$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 1 - 0 = 1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.4 + (0.5 \times 1 \times 0) = 0.4 + 0 = 0.4$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.4 + (0.5 \times 1 \times 1) = 0.4 + 0.5 = 0.9$$

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

1. Get the previous weights

$$w_1 = 0.4$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply third instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.4 + 0 \times 0.9 = 0.4$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 0$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 1 - 0 = 1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.4 + (0.5 \times 1 \times 1) = 0.4 + 0.5 = 0.9$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.9 + (0.5 \times 1 \times 0) = 0.9 + 0 = 0.9$$

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

PROBLEM !

1. Get the previous weights and the threshold

$$w_1 = 0.9$$

$$w_2 = 0.9$$

$$t = 0.5$$

$$\alpha = 0.5$$

2. Apply second instance to the perceptron

$$\sum = x_1 \times w_1 + x_2 \times w_2 = 1 \times 0.9 + 1 \times 0.9 = 1.8$$

3. Check if the output is higher than the threshold: if no, $\hat{y} = 0$, otherwise, $\hat{y} = 1$

$$\hat{y} = 1$$

4. If $y = \hat{y}$, don't adjust the weights; otherwise adjust those based on the error

$$\varepsilon = \text{actual} - \text{prediction} = 0 - 1 = -1$$

$$w_1 = w_1 + (\alpha \times \varepsilon \times x_1) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

$$w_2 = w_2 + (\alpha \times \varepsilon \times x_2) = 0.9 + (0.5 \times -1 \times 1) = 0.9 - 0.5 = 0.4$$

1.2. Neural Networks

The perceptron – An example step by step: XOR Problem

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

PROBLEM !

How to solve this?



Multilayer Perceptron (MLP)



02.

Hyperparameters in Neural Networks

2. Hyperparameters in NN

Introduction



**What are
hyperparameters
in Neural
Networks?**

Parameters in NN that are not learned directly from the data during training, but instead are set previously.

2.1. Hyperparameters in NN

Learning Rate

The learning rate

Determines the step size taken during optimization in the direction of minimizing the loss function.

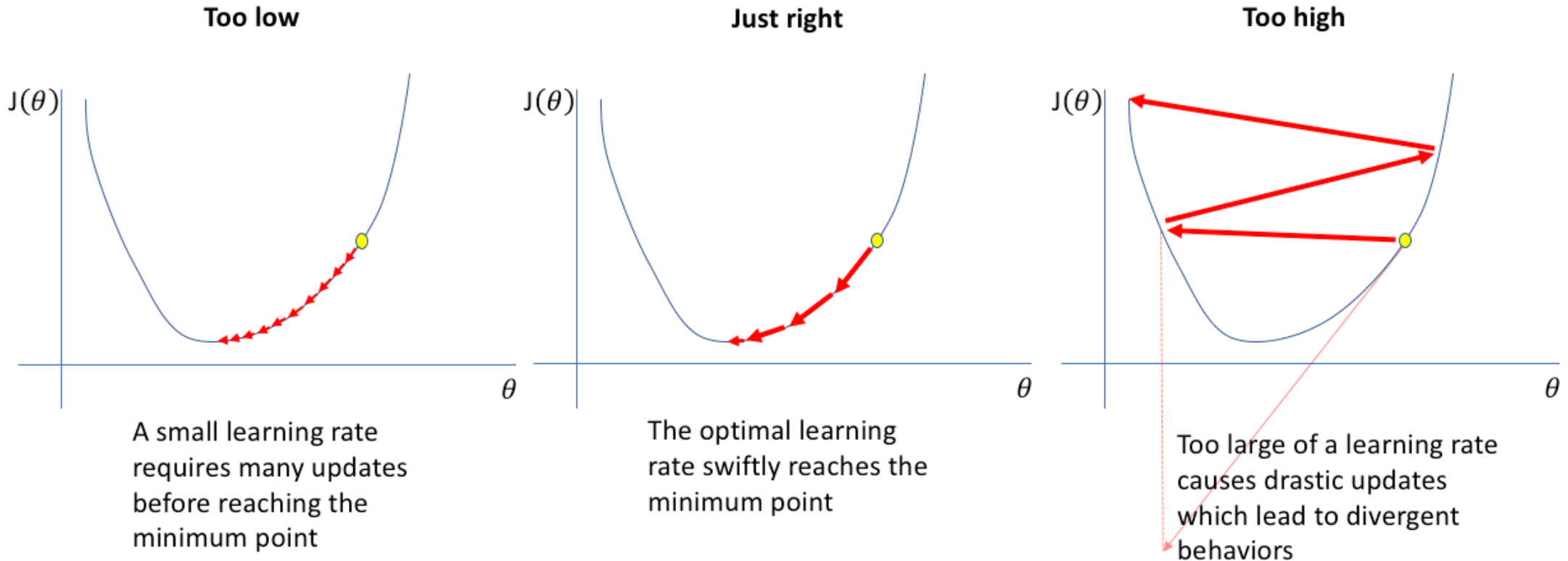
A suitable learning rate can help the model converge faster and improve its accuracy:

- If the learning rate is too high, the model may fail to converge;
- if it is too low, the model may converge slowly or get stuck in a local minimum.

- **The learning rate initialization**
- **The learning rate decay**

2.1. Hyperparameters in NN

Learning Rate



2.1. Hyperparameters in NN

Learning Rate

How to decrease the learning rate?

- *Constant*
 - The learning rate is constant in all iterations
- *Invscaling*

$$LR = \frac{LR_rate_init}{t^{power_t}}, \text{ where } t \text{ stands for step or epoch}$$

- *Adaptive*
 - The learning rate keeps constant as long as training loss keeps decreasing for a specific threshold
 - In Sklearn, if the loss do not decrease by at least a specific threshold (*tol*) for two consecutive epochs, the current learning rate is divided by 5
- ...

2.2. Hyperparameters in NN

Activation Function

Activation Function

- **Transforms the inputs in outputs**
 - Activation functions introduce non-linearity into the output of a neural network's neurons, which is essential for the network to learn complex mappings between input and output data.
- They need to have a **squashing effect**
 - Improve numerical calculations
- The choice of the activation function is generally **problem-dependent**

2.2. Hyperparameters in NN

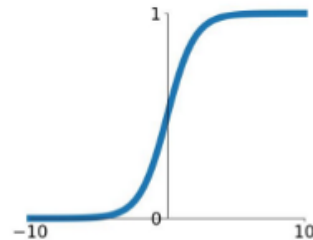
Activation Function

Activation Function

Activation functions introduce non-linearity into the output of a neural network's neurons, which is essential for the network to learn complex mappings between input and output data.

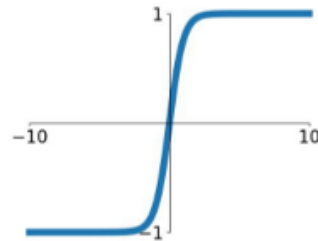
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



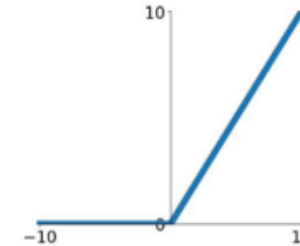
tanh

$$\tanh(x)$$



ReLU

$$\max(0, x)$$



2.3. Hyperparameters in NN

Batches, Iterations and Epochs

What is a batch?

- A set of input data samples processed together in a single forward pass through the NN.
- A larger batch size result in faster training times and better generalization performance but requires more memory and computational resources.

What is an epoch?

- The number of times the entire training dataset is passed forward and backward through the neural network.
- An epoch is complete when all the training samples have been seen by the model once.

2.3. Hyperparameters in NN

Batches, Iterations and Epochs

What is an iteration?

- The number of batches of data that the model goes through during one epoch.
- In other words, an iteration is the process of passing a batch of training data through the model, calculating the loss, and updating the model parameters using an optimization algorithm such as stochastic gradient descent.
- **Example:** if we have 1,000 training samples and a batch size of 100, one epoch consists of 10 iterations. In each iteration, the model updates its weights based on the gradients calculated from the 100 training samples in the batch.

2.4. Hyperparameters in NN

The optimization algorithm

- Mathematical techniques used to optimize the performance of neural networks, i.e., to **minimize the difference between the predicted outputs and the actual outputs**
- In SKLearn:
 - **Gradient Descent:** calculates the derivatives of the loss function with respect to the weights and adjusts the weights iteratively in the direction of steepest descent of the derivatives to minimize the prediction error.
 - **Adam:** Newer optimization algorithm that combines the concept of momentum and adaptive learning rates to achieve faster convergence.
 - ...



03.

The perceptron - Example

3. The perceptron

Example

ID	Weight	Softness	Purr/ min	Bark/ min	Tail Length	Target
1	0.2	0.8	0.3	0.0	0.7	1
2	0.3	0.7	0.2	0.0	0.4	1
3	0.6	0.1	0.0	0.4	0.7	0
4	0.7	0.2	0.0	0.5	0.8	0
5	0.2	0.8	0.3	0.0	0.6	1

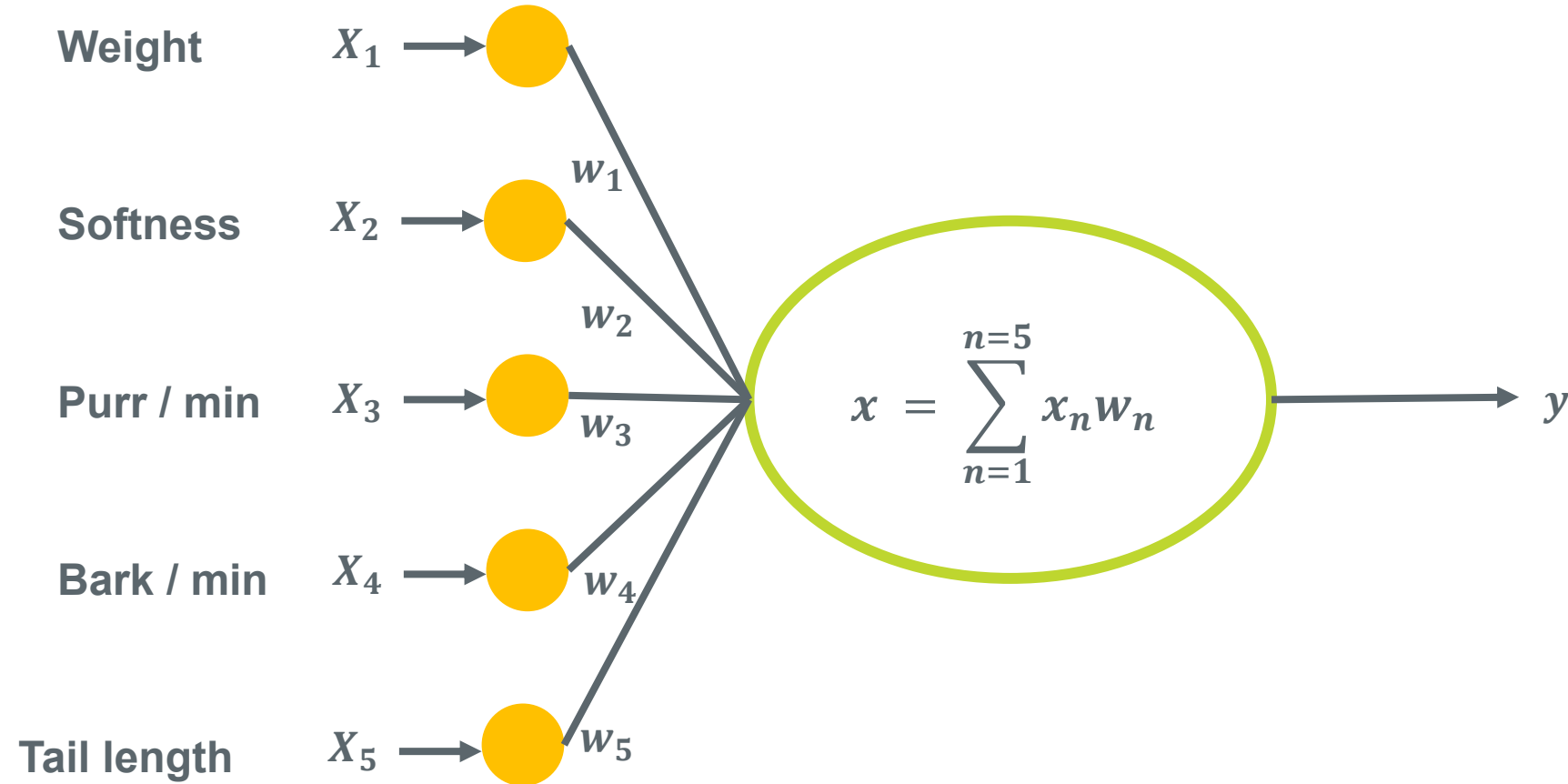
Data has been:

- Preprocessed
- Normalized to the appropriate scales
- **Hyperparameters:**
 - *Activation Function* – Identity
 - *Initial LR*: 0.5
 - *LR decay*: 0.05 at each iteration
 - *Batch size*: 1

The output will be 0 for dog and 1 for cat

3. The perceptron

Example



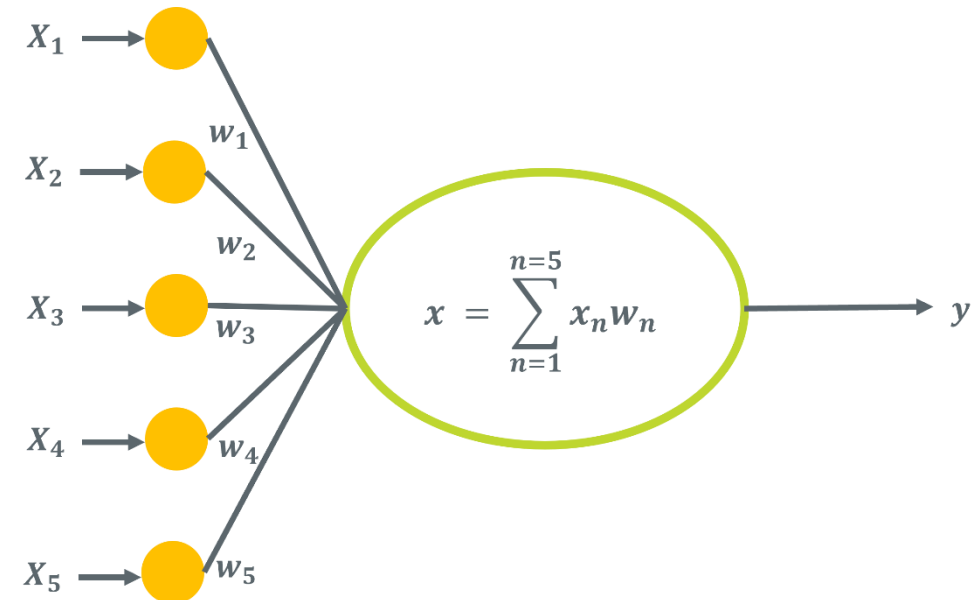
3. The perceptron

Example

ID	Weight	Softness	Purr/mi n	Bark/mi n	Tail Length
1	0.2	0.8	0.3	0.0	0.7

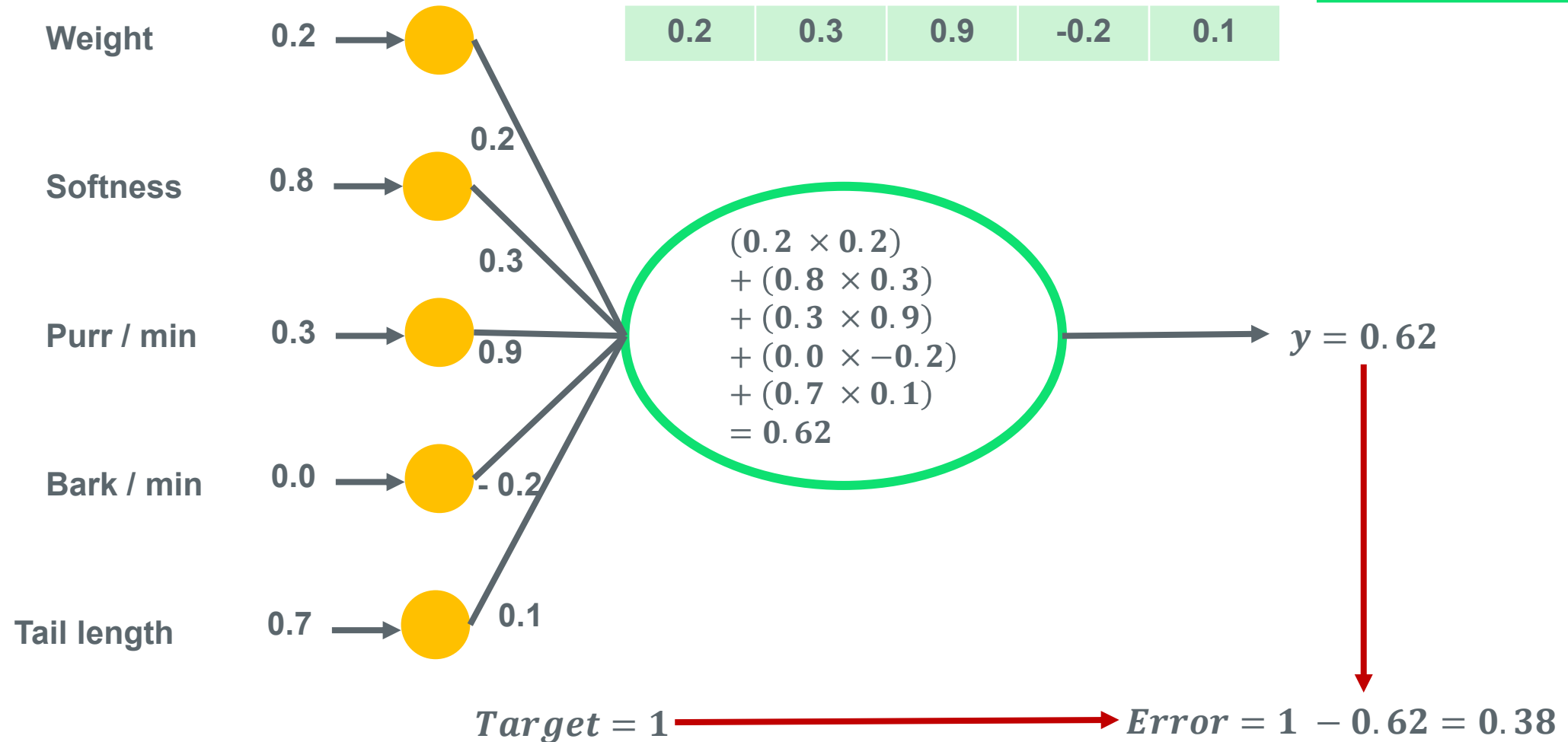
w_1	w_2	w_3	w_4	w_5
0.2	0.3	0.9	-0.2	0.1

Learning rate	0.5
---------------	-----



3. The perceptron

Example

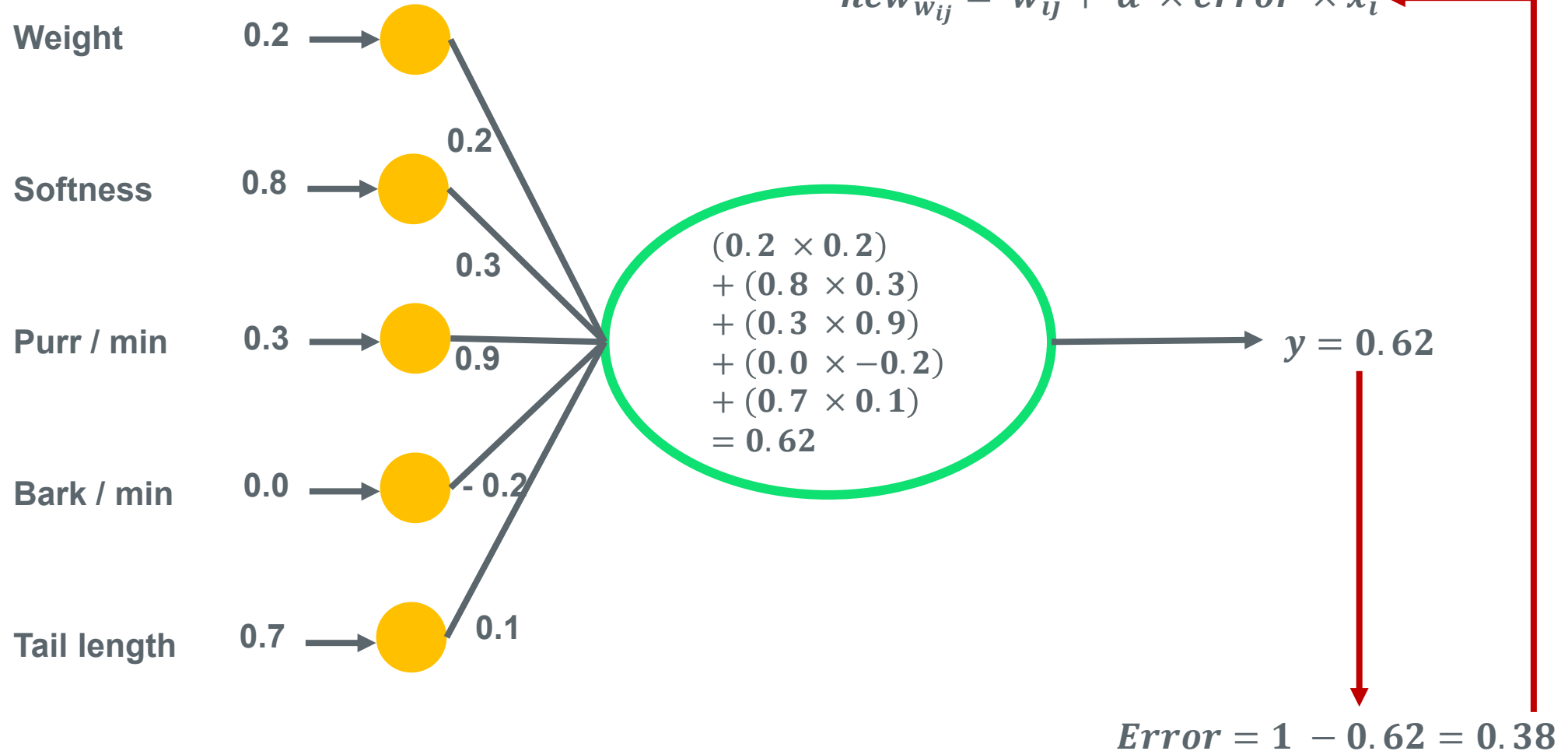


3. The perceptron

Example

w_1	w_2	w_3	w_4	w_5
0.2	0.3	0.9	-0.2	0.1

Learning rate	0.5
---------------	-----

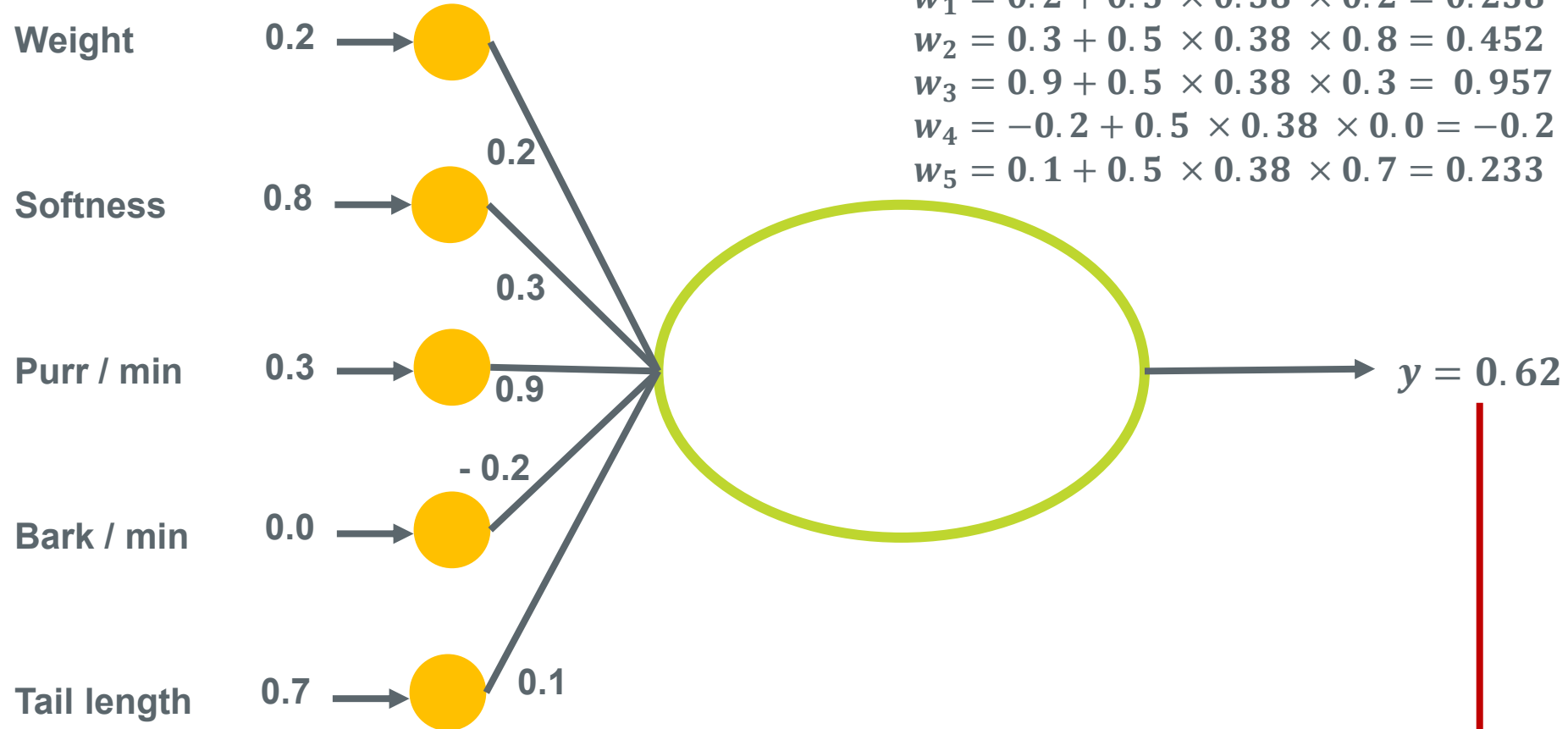


3. The perceptron

Example

w_1	w_2	w_3	w_4	w_5
0.2	0.3	0.9	-0.2	0.1

Learning rate	0.5
---------------	-----



$$new_{w_{ij}} = w_{ij} + \alpha \times error \times x_i$$

$$w_1 = 0.2 + 0.5 \times 0.38 \times 0.2 = 0.238$$

$$w_2 = 0.3 + 0.5 \times 0.38 \times 0.8 = 0.452$$

$$w_3 = 0.9 + 0.5 \times 0.38 \times 0.3 = 0.957$$

$$w_4 = -0.2 + 0.5 \times 0.38 \times 0.0 = -0.2$$

$$w_5 = 0.1 + 0.5 \times 0.38 \times 0.7 = 0.233$$

$$Error = 1 - 0.62 = 0.38$$

3. The perceptron

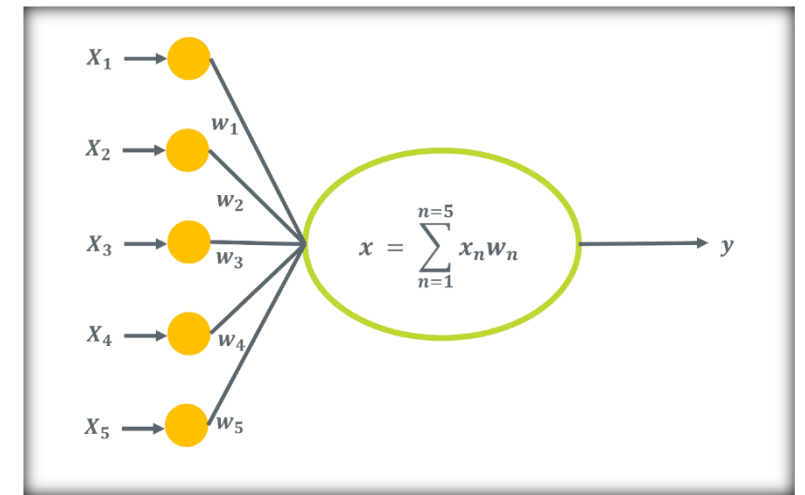
Example



3. The perceptron

Example

- 2 epochs
- Learning rate initialization: 0.5
- Learning rate decay: 0.05
- Initial weights: [0.2,0.3, 0.9, -0.2, 0.1]

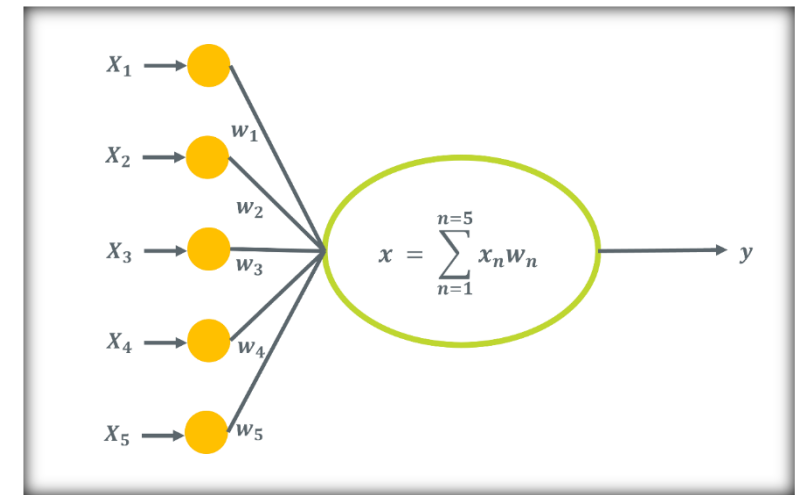


Iter	x1	x2	x3	x4	x5	t	Prediction	Error	w1	w2	w3	w4	w5	L.R.
									0.200	0.300	0.900	-0.200	0.100	0.500
Iter1	0.2	0.8	0.3	0.0	0.7	1	0.620	0.380	0.238	0.452	0.957	-0.200	0.233	0.450
Iter2	0.3	0.7	0.2	0.0	0.4	1	0.672	0.328	0.282	0.555	0.986	-0.200	0.292	0.400
Iter3	0.6	0.1	0.0	0.4	0.7	0	0.349	-0.349	0.198	0.541	0.986	-0.256	0.194	0.350
Iter4	0.7	0.2	0.0	0.5	0.8	0	0.275	-0.275	0.131	0.522	0.986	-0.304	0.117	0.300
Iter5	0.2	0.8	0.3	0.0	0.6	1	0.810	0.190	0.143	0.568	1.004	-0.304	0.151	0.250
Iter6	0.2	0.8	0.3	0.0	0.7	1	0.890	0.110	0.148	0.590	1.012	-0.304	0.171	0.200
Iter7	0.3	0.7	0.2	0.0	0.4	1	0.728	0.272	0.164	0.628	1.023	-0.304	0.193	0.150
Iter8	0.6	0.1	0.0	0.4	0.7	0	0.175	-0.175	0.149	0.625	1.023	-0.314	0.174	0.100
Iter9	0.7	0.2	0.0	0.5	0.8	0	0.211	-0.211	0.134	0.621	1.023	-0.325	0.157	0.050
Iter10	0.2	0.8	0.3	0.0	0.6	1	0.925	0.075	0.135	0.624	1.024	-0.325	0.160	0.000

3. The perceptron

Example

w1	w2	w3	w4	w5
0.135	0.624	1.024	-0.325	0.160



ID	Weight	Softness	Purr/min	Bark/min	Tail Length	Target
1000	0.5	0.6	0.1	0.6	0.4	?

$0.5 * 0.135 + 0.6 * 0.624 + 0.1 * 1.024 + 0.6 * -0.325 + 0.4 * 0.160 = 0.413$

0 for dog
1 for cat

It is a dog!



04.

Multilayer Perceptron

4. Hyperparameters in NN

Introduction



**What is a
Multilayer
Perceptron
(MLP)?**

A feedforward artificial neural network with multiple layers of neurons, capable of modelling complex non-linear relationships between inputs and outputs.

4.1. Multilayer Perceptron

Advantages of MLP neural networks

- Learn automatically
- **Data-driven**
- Especially useful when **we know little about the phenomena** we are trying to model
- Do **non-linear interpolation**
- **Universal approximators**
 - Systems that can approximate functions, whatever complex the shape of this function may be

4.2. Multilayer Perceptron

How to adjust the weights?

How can we define the weights?

General Idea:

- The synapses that help achieving good results should be reinforced
- The synapses that lead to unwanted results should be weakened

4.2. Multilayer Perceptron

How to adjust the weights?

How can we define the weights?

The training algorithm

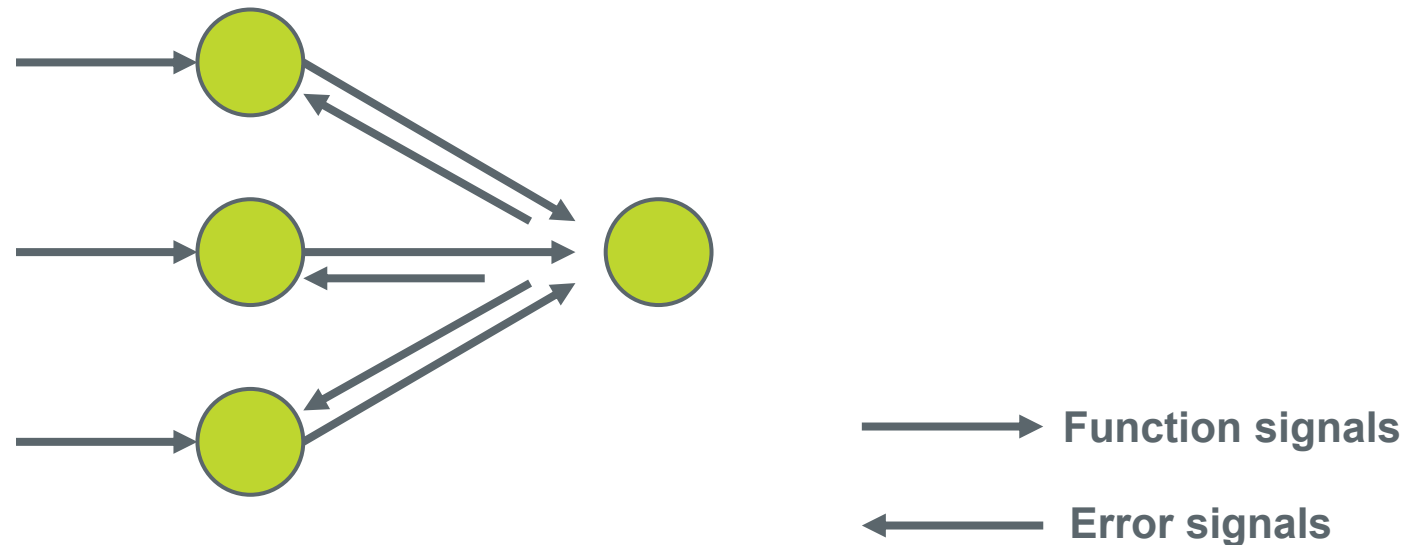
- Adjusts the weights in the network
- It distributes the responsibility for the error measured in the output
- The objective is to generalize knowledge
- Adjusts the synaptic weights in such a way that, given a set of inputs, the desired result is achieved
- The idea: **trial and error**
 - Randomly initialize the weights
 - *If the result is good* – Big luck! Stop!
 - *If the result is wrong* – Do a small adjustment!

4.2. Multilayer Perceptron

How to adjust the weights?

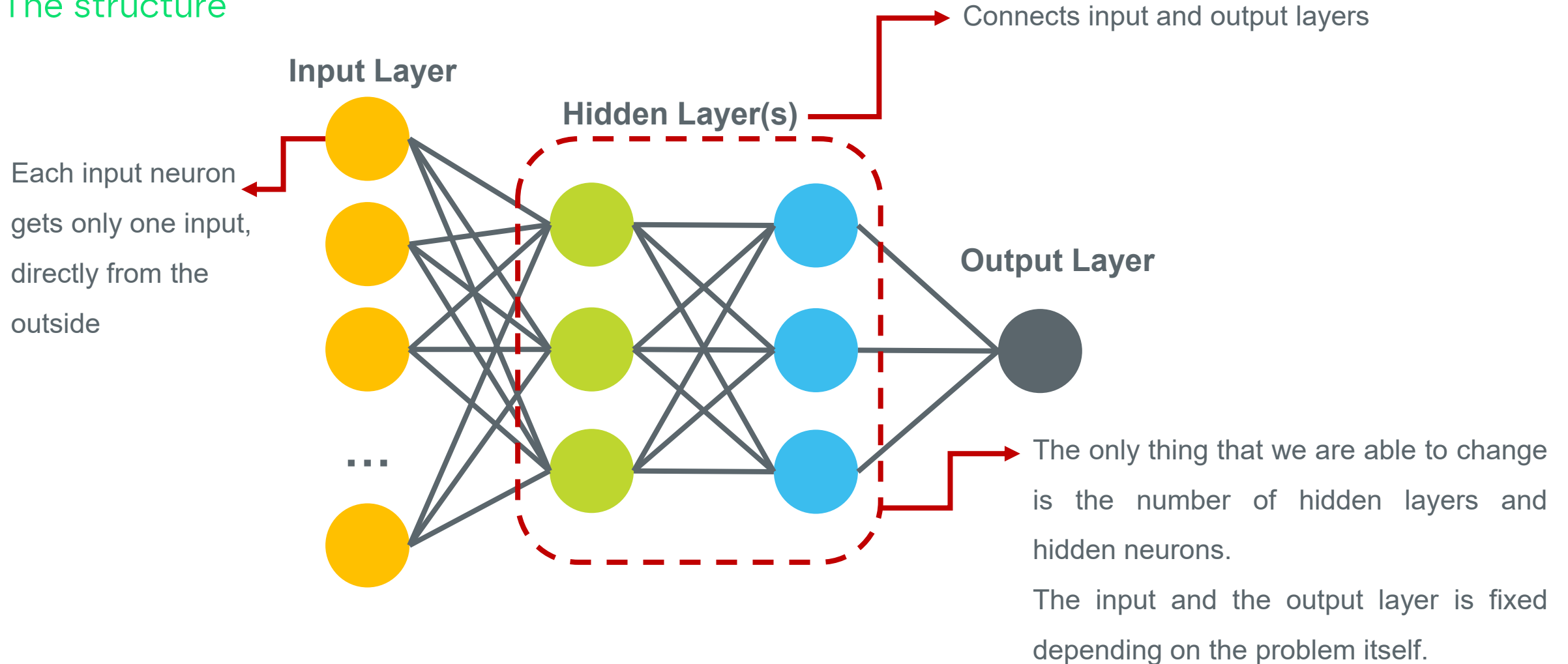
How can we define the weights?

- Function signals which propagate from the input neurons to the output neurons
- Error signals which propagate from the output neurons to the input neurons (layer by layer)



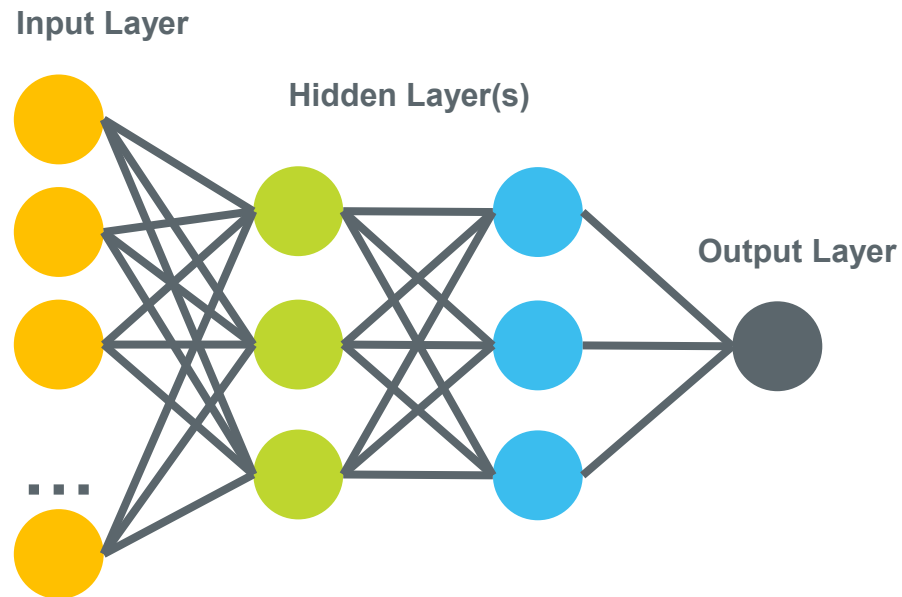
4.3. Multilayer Perceptron

The structure



4.3. Multilayer Perceptron

The structure



Input Layer

- Introduces the input values in the network
- No activation function or any kind of processing

Hidden Layer

- Classifies the characteristics
- Two hidden layers are enough to deal with any problem no matter how complex it may be

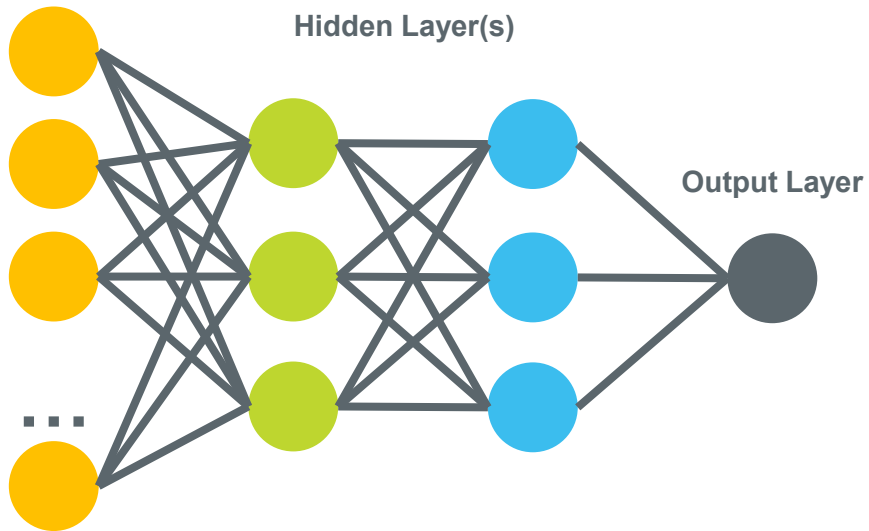
Output Layer

- Same functionality as the hidden layer
- The outputs are passed outside the network

4.4. Multilayer Perceptron

The backpropagation

Input Layer

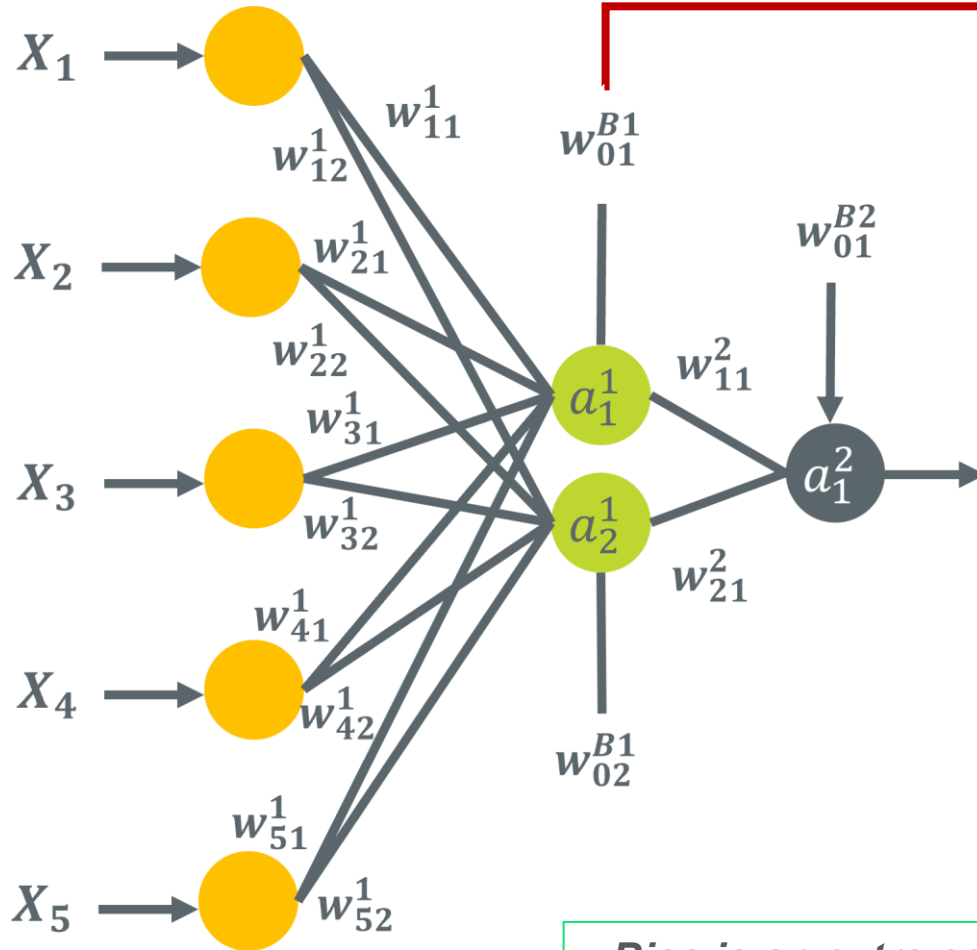


What is backpropagation?

- A training algorithm which allows the adjustment of the weights in a multi-layer feedforward neural network
- Theoretically, it can adjust any function (it can map input-output) no matter how complex
- It can provide solution to linearly inseparable problems
- The use of **derivatives** help adjust the weights and biases of the network during training to minimize the error between predicted and actual outputs (out of topic)

4.5. Multilayer Perceptron

The role of bias



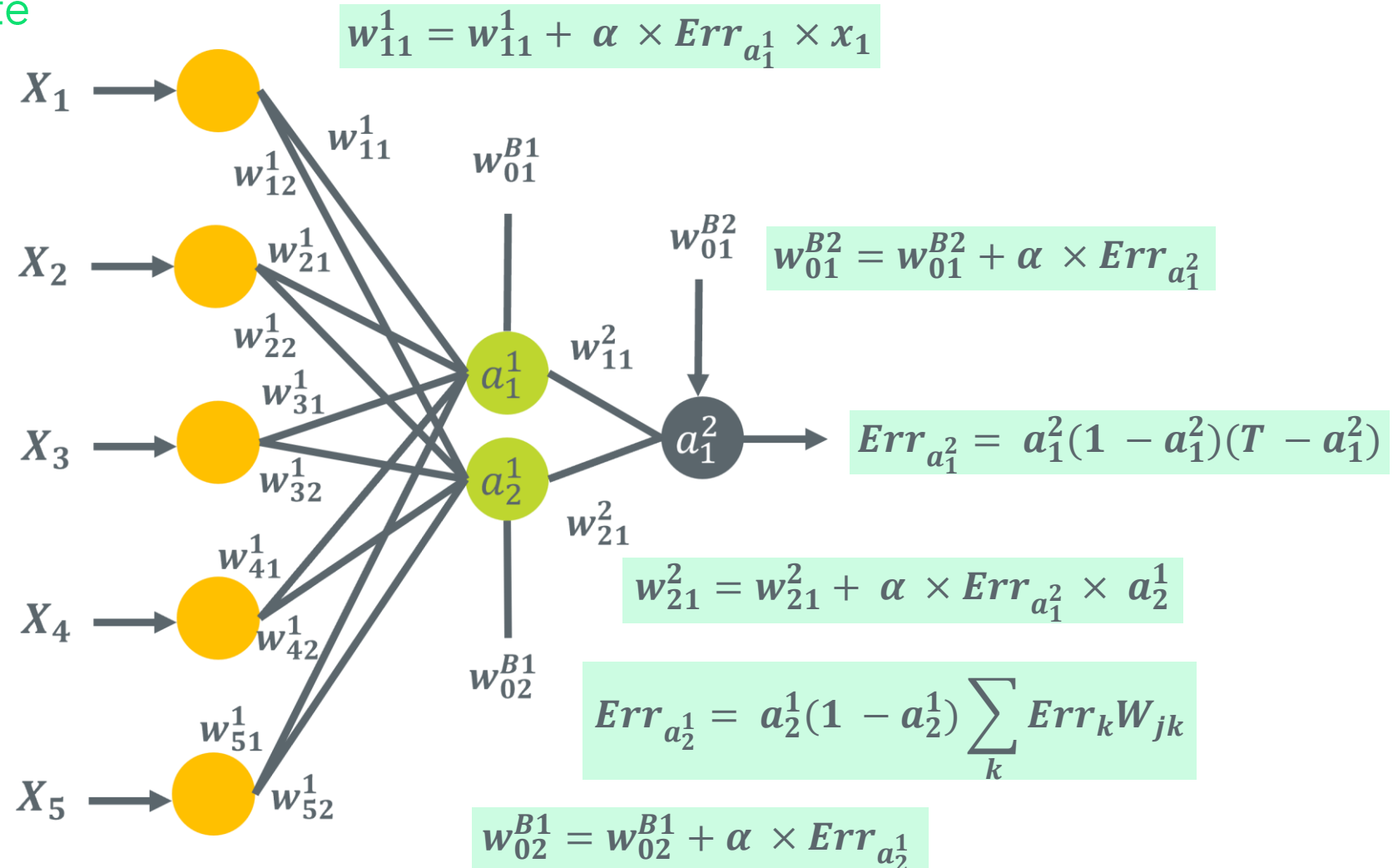
What is the bias?

- It is an **additional constant input** provided to each neuron, allowing the network to model more complex functions and improve its ability to fit the training data
- They provide the network with **additional degrees of freedom** to model complex functions beyond what is possible with only weights
- They **help to ensure that each neuron has a non-zero output** even when all its inputs are zero
- This **can improve the accuracy and performance** of the network by allowing it to better fit the training data and generalize to new data

Bias is an extra adjustable value that helps a neuron shift its output and make better decisions, even when inputs are small or zero.

4.6. Multilayer Perceptron

Error and weight update



4.7. Multilayer Perceptron

An example

ID	Weight	Softness	Purr/min	Bark/min	Tail Length	Target
1	0.2	0.8	0.3	0.0	0.7	1
2	0.3	0.7	0.2	0.0	0.4	1
3	0.6	0.1	0.0	0.4	0.7	0
4	0.7	0.2	0.0	0.5	0.8	0
5	0.2	0.8	0.3	0.0	0.6	1

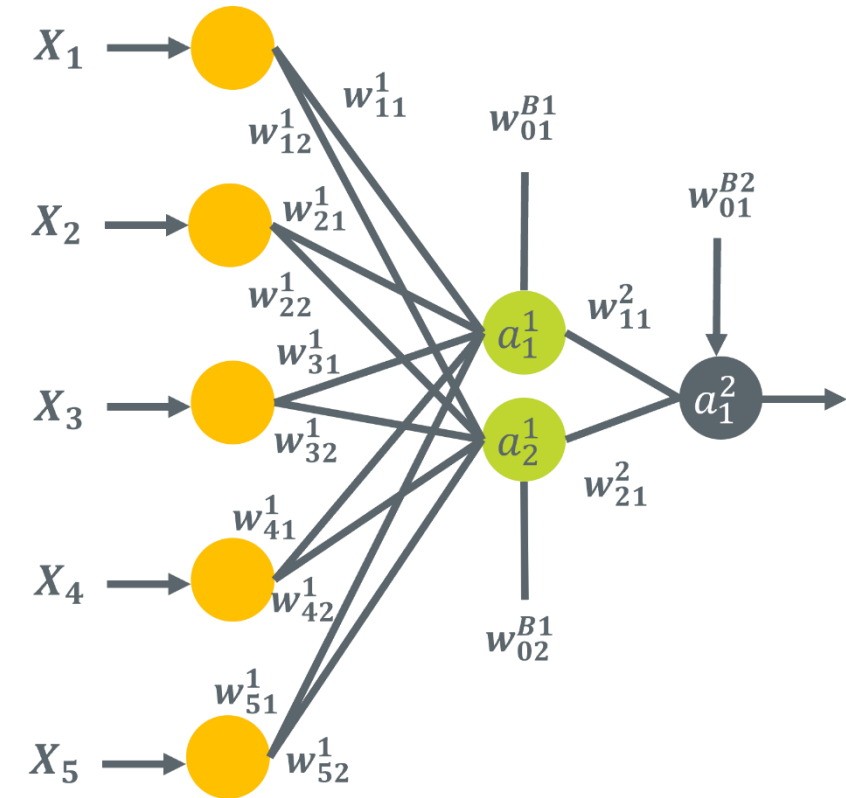
Hyperparameters

- **Structure:** One hidden layer with two neurons
- **Activation function:** Sigmoid
- **Initial Learning rate:** 0.5
- **Learning rate decay:** 0.05
- **Batch size:** 1

4.7. Multilayer Perceptron

An example

ID	Weight	Softness	Purr/ min	Bark/ min	Tail Length	Target
1	0.2	0.8	0.3	0.0	0.7	1
2	0.3	0.7	0.2	0.0	0.4	1
3	0.6	0.1	0.0	0.4	0.7	0
4	0.7	0.2	0.0	0.5	0.8	0
5	0.2	0.8	0.3	0.0	0.6	1



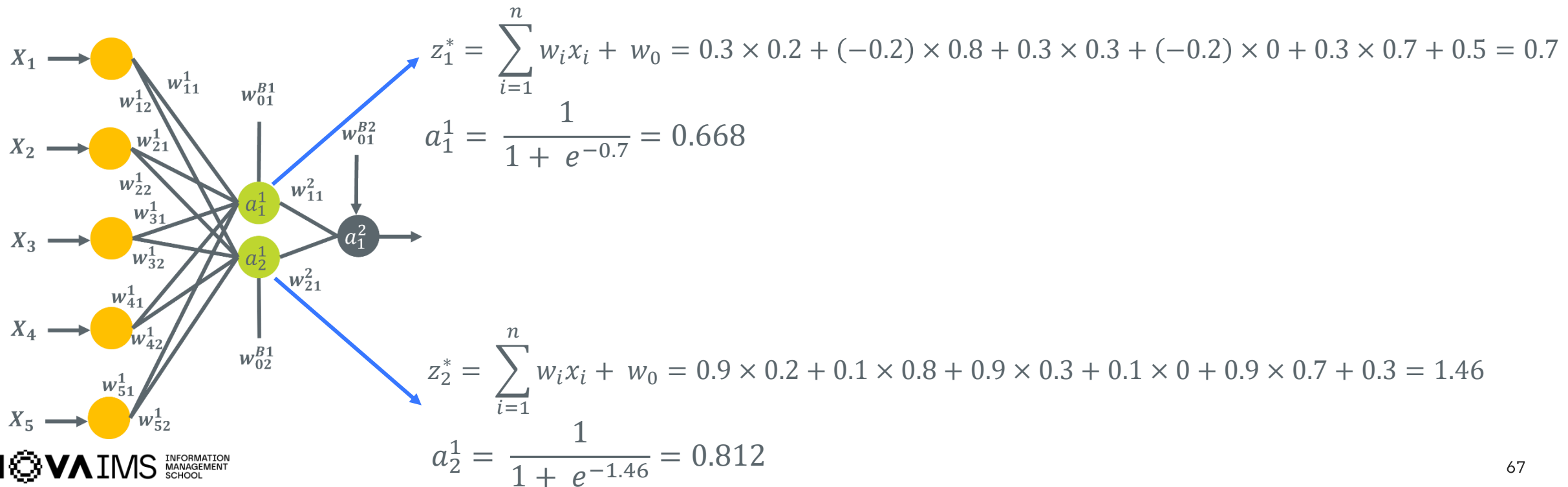
w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{21}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2

4.7. Multilayer Perceptron

An example

ID	X_1	X_2	X_3	X_4	X_5	t
1	0.2	0.8	0.3	0.0	0.7	1

w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{21}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2

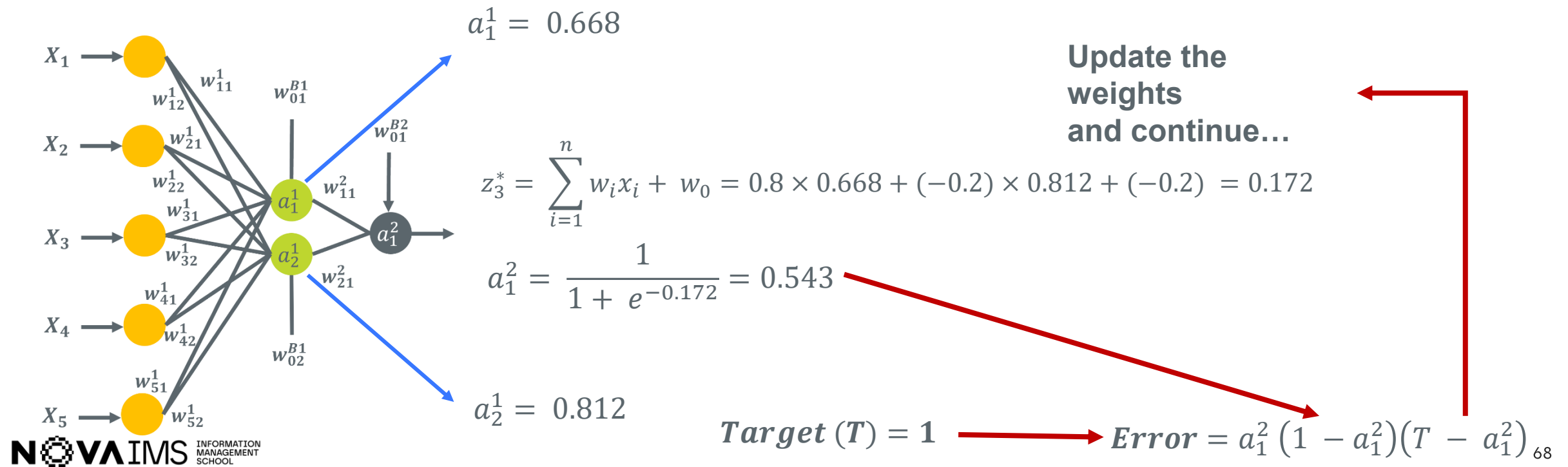


4.7. Multilayer Perceptron

An example

ID	X_1	X_2	X_3	X_4	X_5	t
1	0.2	0.8	0.3	0.0	0.7	1

w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{21}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2

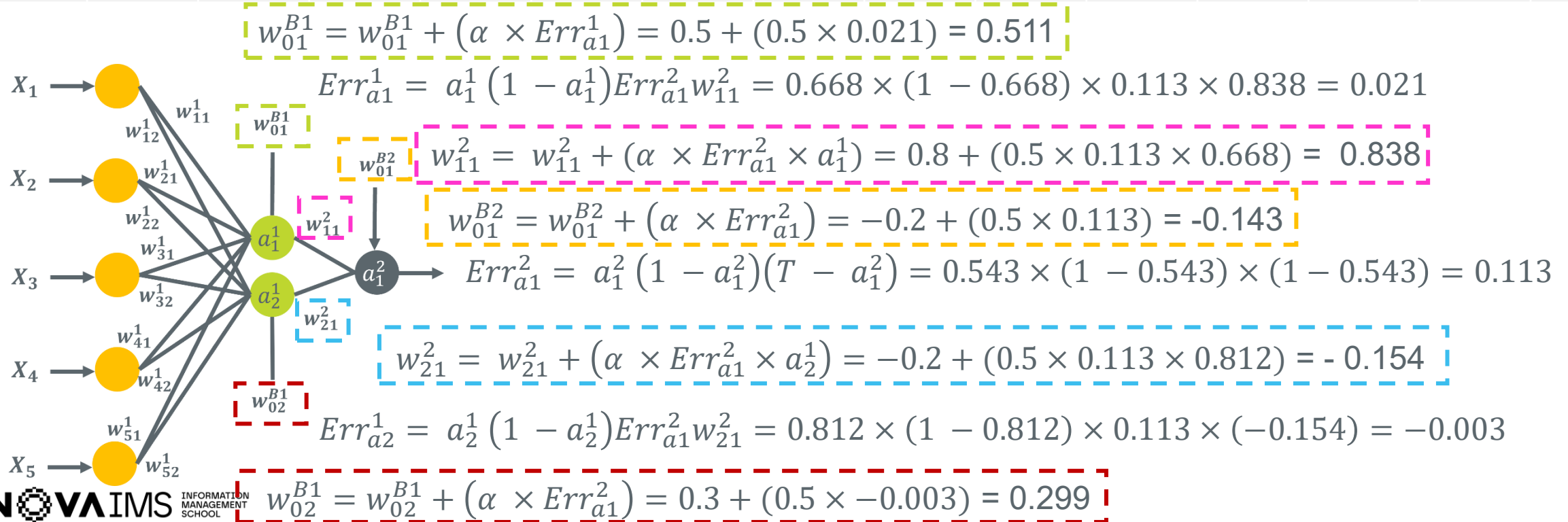


4.7. Multilayer Perceptron

An example

ID	X_1	X_2	X_3	X_4	X_5	t
1	0.2	0.8	0.3	0.0	0.7	1

w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{21}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2

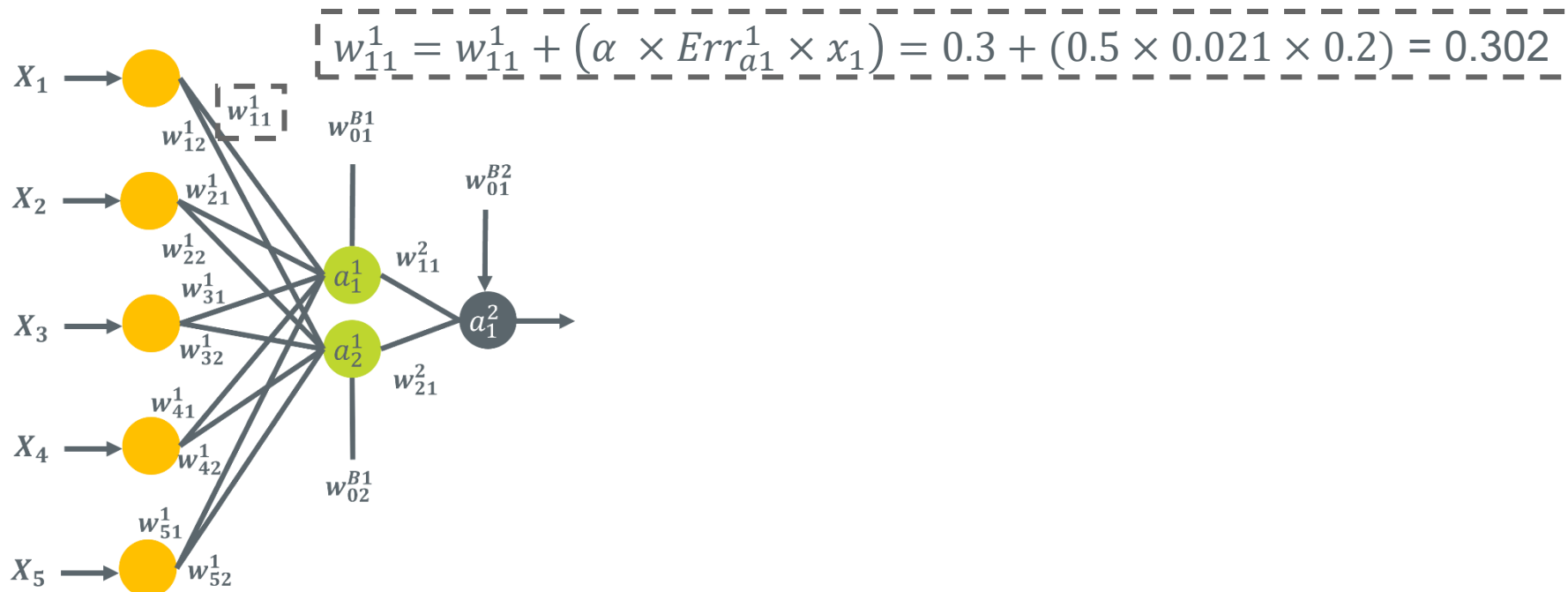


4.7. Multilayer Perceptron

An example

ID	X_1	X_2	X_3	X_4	X_5	t
1	0.2	0.8	0.3	0.0	0.7	1

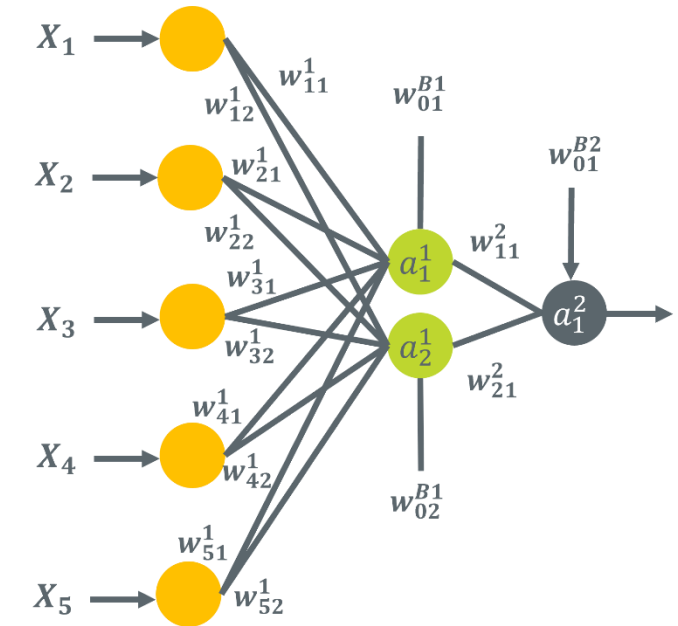
w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{12}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2



4.7. Multilayer Perceptron

An example

ID	X_1	X_2	X_3	X_4	X_5	t
1	0.2	0.8	0.3	0.0	0.7	1



w_{11}^1	w_{12}^1	w_{21}^1	w_{22}^1	w_{31}^1	w_{32}^1	w_{41}^1	w_{42}^1	w_{51}^1	w_{52}^1	w_{01}^{B1}	w_{02}^{B1}	w_{11}^2	w_{12}^2	w_{01}^{B2}
0.3	0.9	-0.2	0.1	0.3	0.9	-0.2	0.1	0.3	0.9	0.5	0.3	0.8	-0.2	-0.2
0.302	0.900	-0.192	0.099	0.303	0.900	-0.200	0.100	0.307	0.899	0.511	0.299	0.838	-0.154	-0.143

z_1^1	z_2^1	a_1^1	a_2^1	z_1^2	a_1^2
0.700	1.460	0.668	0.812	0.172	0.543

$Err_{a_1^1}$	$Err_{a_2^1}$	$Err_{a_1^2}$
0.021	-0.003	0.113

4.7. Multilayer Perceptron

An example

Iter	X1	X2	X3	X4	X5	T	w_11^1	w_12^1	w_21^1	w_22^1	w_31^1	w_32^1	w_41^1	w_42^1	w_51^1	w_52^1	w_01^1B1	w_02^1B1	z_1^1	z_2^1	a_1^1	a_2^1	w_11^2	w_21^2	w_01^2B2	z_1^2	a_1^2	LR	Err_a_1^1	Err_a_2^1	Err_a_1^2
1	0	1	0	0	1	1	0.300	0.900	-0.200	0.100	0.300	0.900	-0.200	0.100	0.300	0.900	0.500	0.300					0.800	-0.200	-0.200			0.500			
FORWARD																			0.700	1.460	0.668	0.812				0.172	0.543				
BACKWARD							0.302	0.900	-0.192	0.099	0.303	0.900	-0.200	0.100	0.307	0.899	0.511	0.299					0.838	-0.154	-0.143				0.021	-0.003	0.113
2	0	1	0	0	0	1	0.302	0.900	-0.192	0.099	0.303	0.900	-0.200	0.100	0.307	0.899	0.511	0.299					0.838	-0.154	-0.143			0.450			
FORWARD																			0.651	1.177	0.657	0.764				0.290	0.572				
BACKWARD							0.305	0.899	-0.185	0.098	0.305	0.899	-0.200	0.100	0.311	0.899	0.520	0.298					0.869	-0.118	-0.096				0.021	-0.002	0.105
3	1	0	0	0	1	0	0.305	0.899	-0.185	0.098	0.305	0.899	-0.200	0.100	0.311	0.899	0.520	0.298					0.869	-0.118	-0.096			0.400			
FORWARD																			0.822	1.516	0.695	0.820				0.411	0.601				
BACKWARD							0.299	0.900	-0.186	0.098	0.305	0.899	-0.204	0.101	0.304	0.900	0.510	0.299					0.829	-0.165	-0.154				-0.025	0.004	-0.144
4	1	0	0	1	1	0	0.299	0.900	-0.186	0.098	0.305	0.899	-0.204	0.101	0.304	0.900	0.510	0.299					0.829	-0.165	-0.154			0.350			
FORWARD																			0.823	1.719	0.695	0.848				0.282	0.570				
BACKWARD							0.293	0.901	-0.188	0.099	0.305	0.899	-0.208	0.101	0.297	0.901	0.501	0.300					0.795	-0.207	-0.203				-0.024	0.004	-0.140
5	0	1	0	0	1	1	0.293	0.901	-0.188	0.099	0.305	0.899	-0.208	0.101	0.297	0.901	0.501	0.300					0.795	-0.207	-0.203			0.300			
FORWARD																			0.680	1.370	0.664	0.797				0.160	0.540				
BACKWARD							0.294	0.901	-0.183	0.098	0.307	0.899	-0.208	0.101	0.301	0.900	0.508	0.299					0.818	-0.179	-0.168				0.021	-0.003	0.114
6	0	1	0	0	1	1	0.294	0.901	-0.183	0.098	0.307	0.899	-0.208	0.101	0.301	0.900	0.508	0.299					0.818	-0.179	-0.168			0.250			
FORWARD																			0.723	1.458	0.673	0.811				0.237	0.559				
BACKWARD							0.295	0.901	-0.179	0.097	0.308	0.899	-0.208	0.101	0.305	0.900	0.513	0.299					0.836	-0.157	-0.141				0.020	-0.003	0.109
7	0	1	0	0	0	1	0.295	0.901	-0.179	0.097	0.308	0.899	-0.208	0.101	0.305	0.900	0.513	0.299					0.836	-0.157	-0.141			0.200			
FORWARD																			0.660	1.177	0.659	0.764				0.290	0.572				
BACKWARD							0.296	0.901	-0.176	0.097	0.309	0.899	-0.208	0.101	0.306	0.899	0.517	0.298					0.850	-0.141	-0.120				0.020	-0.003	0.105
8	1	0	0	0	1	0	0.296	0.901	-0.176	0.097	0.309	0.899	-0.208	0.101	0.306	0.899	0.517	0.298					0.850	-0.141	-0.120			0.150			
FORWARD																			0.808	1.518	0.692	0.820				0.352	0.587				
BACKWARD							0.294	0.901	-0.176	0.097	0.309	0.899	-0.210	0.101	0.304	0.900	0.513	0.299					0.835	-0.159	-0.142				-0.025	0.003	-0.142
9	1	0	0	1	1	0	0.294	0.901	-0.176	0.097	0.309	0.899	-0.210	0.101	0.304	0.900	0.513	0.299					0.835	-0.159	-0.142			0.100			
FORWARD																			0.822	1.719	0.695	0.848				0.304	0.575				
BACKWARD							0.292	0.901	-0.177	0.097	0.309	0.899	-0.211	0.102	0.302	0.900	0.510	0.299					0.825	-0.171	-0.156				-0.025	0.003	-0.141
10	0	1	0	0	1	1	0.292	0.901	-0.177	0.097	0.309	0.899	-0.211	0.102	0.302	0.900	0.510	0.299					0.825	-0.171	-0.156			0.050			
FORWARD																			0.701	1.367	0.668	0.797				0.260	0.565				
BACKWARD							0.293	0.901	-0.176	0.097	0.309	0.899	-0.211	0.102	0.302	0.900	0.511	0.299					0.829	-0.166	-0.150				0.020	-0.003	0.107

Obrigada!

Thank you!

Acreditações e Certificações da NOVA IMS



Cofinanciado por

